

Compiler Design: Assignment Problems

22 Project Problems and Submission Guidelines

Each group chooses **one** of the 22 problems below and builds a working tool with a complete README. All work is submitted in the course repository <https://github.com/venkyjntu-git/compiler-design-assignments-2026-27> as described in Section 8. Submission deadline: **28 October 2026, 23:59 IST**.

Contents

1	Overview of the problems	3
2	Track A: Lexing and parsing	3
2.1	Problem 1: Interactive LL(1) Grammar Workbench	3
2.2	Problem 2: LR(1) and LALR(1) Parser Workbench with Conflict Analyzer	4
2.3	Problem 3: Syntax Error Recovery Simulator	4
2.4	Problem 4: Lexer Generator: Regular Expressions to Minimized DFA	5
2.5	Problem 5: Mini Parser Generator (Grammar to Parser Source Code)	5
2.6	Problem 6: Attribute Grammar Evaluator	6
3	Track B: Domain-specific languages	6
3.1	Problem 7: Mini SQL Query Compiler and Optimizer	6
3.2	Problem 8: Tensor Expression DSL Compiled to Optimized Loop Nests	7
4	Track C: Optimization and back end	8
4.1	Problem 9: DAG Construction and Local Optimization of Basic Blocks	8
4.2	Problem 10: Control Flow Graph and Data-Flow Analysis Framework	8
4.3	Problem 11: Register Allocation by Graph Coloring	9
4.4	Problem 12: SSA Construction and SSA-Based Optimization	9
5	Track D: Static analysis and code metrics	10
5.1	Problem 13: Code Complexity and Maintainability Analyzer	10
5.2	Problem 14: AST-Based Code Clone and Plagiarism Detector	11
5.3	Problem 15: AST-Based Security Analyzer with Taint Tracking	11
5.4	Problem 16: AST Query Engine (a Mini CodeQL / Semgrep)	12
5.5	Problem 17: Call Graph Generator and Dead Code Finder	12
6	Track E: Quantum compilation with OpenQASM	13
6.1	Problem 18: OpenQASM Front End: Lexer, Parser and Semantic Checker	13

6.2	Problem 19: Quantum Circuit Optimizer (Peephole and DAG-Based)	13
6.3	Problem 20: Qubit Mapping and Routing on Hardware Coupling Graphs	14
6.4	Problem 21: A Small High-Level Quantum Language Compiled to OpenQASM	14
7	Track F: Visual and educational compiler tooling	15
7.1	Problem 22: Compiler Pipeline Explorer with Animated Lessons and Adaptive Quizzes . . .	15
8	Submission guidelines	16
8.1	General rules	16
8.2	README template	16
8.3	Evaluation rubric	17

1 Overview of the problems

The problems are grouped into six tracks. Each problem connects a syllabus topic to a tool that compiler and developer-tooling engineers actually build, so the finished project also serves as a portfolio and resume item.

#	Problem	Track
1	LL(1) Grammar Workbench	A
2	LR(1)/LALR(1) Workbench and Conflict Analyzer	A
3	Syntax Error Recovery Simulator	A
4	Lexer Generator: Regular Expressions to Minimized DFA	A
5	Mini Parser Generator	A
6	Attribute Grammar Evaluator	A
7	Mini SQL Query Compiler and Optimizer	B
8	Tensor Expression DSL Compiled to Optimized Loop Nests	B
9	DAG Construction and Local Optimization	C
10	CFG and Data-Flow Analysis Framework	C
11	Register Allocation by Graph Coloring	C
12	SSA Construction and SSA-Based Optimization	C
13	Code Complexity and Maintainability Analyzer	D
14	AST-Based Code Clone and Plagiarism Detector	D
15	AST-Based Security Analyzer with Taint Tracking	D
16	AST Query Engine (a Mini CodeQL / Semgrep)	D
17	Call Graph Generator and Dead Code Finder	D
18	OpenQASM Front End: Lexer, Parser and Semantic Checker	E
19	Quantum Circuit Optimizer (Peephole and DAG-Based)	E
20	Qubit Mapping and Routing on Hardware Coupling Graphs	E
21	A Small High-Level Quantum Language Compiled to OpenQASM	E
22	Compiler Pipeline Explorer with Animated Lessons and Quizzes	F

Tracks. A: Lexing and parsing. B: Domain-specific languages. C: Optimization and back end. D: Static analysis and code metrics. E: Quantum compilation with OpenQASM. F: Visual and educational compiler tooling.

Structure of each problem. Every problem gives a *problem statement*, the *goal* (what you should learn), *required features* (needed for full marks), *stretch goals* (bonus), *problem-specific deliverables* (in addition to the common rules in Section 8) and a *career relevance* line you can adapt for your resume.

2 Track A: Lexing and parsing

2.1 Problem 1: Interactive LL(1) Grammar Workbench

Track: A

Problem statement. Students compute FIRST and FOLLOW sets and LL(1) tables by hand and rarely see why a grammar fails. Build a tool that takes any context-free grammar, transforms it toward LL(1) form, builds the predictive parsing table, and animates the parse of an input string step by step.

Goal. Master FIRST/FOLLOW computation, left-recursion elimination, left factoring and table-driven predictive parsing.

Required features.

- Grammar input in a simple text format ($E \rightarrow E + T \mid T$), with ϵ support.

- Elimination of direct and indirect left recursion, and left factoring, showing the grammar before and after.
- FIRST and FOLLOW sets, with the iteration in which each symbol was added.
- LL(1) table with conflicting cells highlighted, and a verdict (LL(1) or not) with the reason.
- Step-by-step parse trace (stack, input, action) and the resulting parse tree.

Stretch goals. Panic-mode error recovery using synchronizing sets; export of the table to $\text{\LaTeX}$; generation of a recursive-descent parser from the grammar.

Problem-specific deliverables. At least 10 test grammars in `examples/`, including three non-LL(1) grammars with explained conflicts. A web interface is preferred.

Career relevance. “Built a web-based LL(1) parser workbench with grammar transformation and animated parsing.”

2.2 Problem 2: LR(1) and LALR(1) Parser Workbench with Conflict Analyzer

Track: A

Problem statement. Build a tool that constructs the canonical LR(1) item sets, the ACTION and GOTO tables, and the merged LALR(1) tables for a given grammar. It must detect shift-reduce and reduce-reduce conflicts and explain each one with a counterexample input.

Goal. Understand LR(0), SLR(1), LR(1) and LALR(1) construction, and why LALR merging can introduce reduce-reduce conflicts.

Required features.

- Canonical LR(1) collection with closure and goto, drawn as a DFA of item sets.
- ACTION/GOTO tables for SLR(1), LR(1) and LALR(1), with state counts for each.
- Conflict report: state, lookahead, conflicting items, and a shortest input prefix that reaches the conflict.
- Shift-reduce parse trace for an input string, building the parse tree bottom-up.
- Precedence and associativity declarations (as in Yacc) to resolve conflicts, showing which entries changed.

Stretch goals. Side-by-side LL(1) versus LR(1) comparison on the same grammar; import of grammars in Bison format.

Problem-specific deliverables. Include the dangling-else grammar, the classic expression grammar, and one grammar that is LR(1) but not LALR(1), each analysed in the README.

Career relevance. “Implemented canonical LR(1)/LALR(1) table construction and a conflict explainer, similar to Bison’s counterexample feature.”

2.3 Problem 3: Syntax Error Recovery Simulator

Track: A

Problem statement. Real compilers must continue after the first syntax error and report useful messages. Build a parser for a small C-like language that demonstrates and compares panic-mode, phrase-level and error-production recovery.

Goal. Learn how synchronizing tokens, insertion/deletion repairs and error productions affect message quality and cascading errors.

Required features.

- A grammar for a mini language (declarations, assignments, `if`, `while`, blocks, expressions).
- Three recovery strategies selectable by a flag, each producing line and column error messages.
- A benchmark of at least 30 faulty programs, measuring errors reported, spurious (cascading) errors and recovery success for each strategy.
- Repair suggestions such as “missing ‘;’ before ‘while’” where possible.

Stretch goals. Minimum-cost repair using a small edit-distance search; comparison of your messages with GCC or Clang output on equivalent C snippets.

Problem-specific deliverables. A results table in the README comparing the three strategies.

Career relevance. “Designed and benchmarked three syntax error-recovery strategies for a C-like language.”

2.4 Problem 4: Lexer Generator: Regular Expressions to Minimized DFA

Track: A

Problem statement. Build a mini `lex`. It reads token definitions written as regular expressions, converts them to an NFA (Thompson’s construction), then to a DFA (subset construction), minimizes the DFA, and emits a working lexer. Demonstrate it on at least two different languages from separate specification files.

Goal. Connect automata theory to practical scanners, including longest-match and rule-priority behaviour.

Required features.

- Regex parser supporting concatenation, union, Kleene star, plus, optional, character classes and escapes.
- Thompson NFA, subset-construction DFA and Hopcroft minimization, each visualized with Graphviz.
- Maximal munch with rule priority; line and column tracking; error tokens.
- Token specifications for two languages (for example a C subset and a Python subset with indentation tokens).
- The generated lexer as source code, not only an interpreter.

Stretch goals. Compare generated-lexer speed with a hand-written lexer and with Python’s `re`; Unicode support.

Problem-specific deliverables. State counts before and after minimization for each specification, reported in the README.

Career relevance. “Wrote a lexer generator from regular expressions to minimized DFA with code generation.”

2.5 Problem 5: Mini Parser Generator (Grammar to Parser Source Code)

Track: A

Problem statement. Build a small parser generator in the spirit of Yacc or ANTLR. Given an annotated grammar file, it generates the source code of an LL(1) recursive-descent parser or an LALR(1) table-driven parser that builds an AST.

Goal. Understand parser generators as compilers themselves: a grammar is the input language and parser code is the output.

Required features.

- A grammar file format with tokens, rules and semantic actions that construct AST nodes.
- Generation of a standalone parser in at least one target language.
- Grammar checks: undefined symbols, unreachable and unproductive nonterminals, conflicts.
- Demonstration: a generated parser for a calculator language and one for JSON, tested on real inputs.

Stretch goals. Two target languages; self-hosting, where the generator’s own grammar file is parsed by a generated parser.

Problem-specific deliverables. Generated code committed under `examples/generated/`. This is the hardest problem in Track A and is intended for a pair.

Career relevance. “Built a parser generator that emits AST-building parsers from annotated grammars.”

2.6 Problem 6: Attribute Grammar Evaluator

Track: A

Problem statement. Build an engine that takes an attribute grammar (productions plus semantic rules over synthesized and inherited attributes), parses an input, builds the dependency graph over the parse tree, and evaluates attributes in a valid topological order.

Goal. Master syntax-directed definitions, S-attributed and L-attributed grammars, and dependency-graph evaluation.

Required features.

- An input format for productions with attribute equations.
- Classification of the grammar as S-attributed, L-attributed or neither.
- Annotated parse tree and dependency-graph visualization, with the evaluation order numbered.
- Cycle detection with a clear error report.
- At least three demonstrations: expression evaluation, type declarations (`int a, b, c`), and the value of a binary number.

Stretch goals. One-pass evaluation during LL parsing for L-attributed grammars; automatic translation of an attribute grammar into a syntax-directed translation scheme.

Problem-specific deliverables. The README must show one dependency graph drawn by hand next to your tool’s output.

Career relevance. “Implemented an attribute-grammar evaluator with dependency analysis and visualization.”

3 Track B: Domain-specific languages

Domain-specific languages (DSLs) are small languages built for one job, such as SQL for data or Halide for image processing. Their compilers reuse every classical phase but add domain knowledge in the optimizer, which is where most industrial compiler work happens today.

3.1 Problem 7: Mini SQL Query Compiler and Optimizer

Track: B

Problem statement. A database engine is a compiler: it parses a SQL query, checks it against the schema, turns it into a relational-algebra plan, rewrites the plan into a cheaper equivalent, and executes it. Build this pipeline for a subset of SQL running over CSV tables, and show how much the optimizer improves execution.

Goal. Apply lexing, parsing, semantic analysis and IR rewriting to a real declarative DSL, and learn rule-based and cost-based query optimization.

Required features.

- Lexer and parser for `SELECT` with `FROM`, `WHERE`, inner `JOIN . . . ON`, `GROUP BY`, `HAVING`, `ORDER BY`, `LIMIT`, and the aggregates `COUNT`, `SUM`, `AVG`, `MIN`, `MAX`.
- Semantic analysis against a schema catalog: unknown or ambiguous columns, unknown tables, type errors in predicates, and non-aggregated columns missing from `GROUP BY`.
- Translation to a logical plan (a relational-algebra tree) drawn as a diagram.
- Rewrite rules: constant folding, predicate pushdown, projection pruning, and join reordering guided by a simple cost model using table sizes and selectivity estimates.
- An iterator-model (Volcano) executor over CSV files, and an `EXPLAIN` command that prints the plan before and after optimization with estimated costs.
- Validation: results match SQLite on at least 30 queries, and a table compares running time with and without the optimizer.

Stretch goals. Compile the optimized plan to Python or C source code instead of interpreting it (query compilation, as in HyPer and Umbra); hash joins and simple indexes; comparison with DuckDB’s `EXPLAIN` output on the same queries.

Problem-specific deliverables. The supported SQL grammar in EBNF, a small test database (for example a scaled-down TPC-H or a college-records dataset) with a script to generate it, and the validation and timing tables in the README.

Career relevance. “Built a SQL query compiler with rule- and cost-based optimization, validated against SQLite.”

3.2 Problem 8: Tensor Expression DSL Compiled to Optimized Loop Nests

Track: B

Problem statement. Machine-learning compilers such as Halide, TVM and MLIR separate *what* to compute from *how* to compute it. Design a small DSL in which users write tensor computations in index notation, for example `C[i, j] = sum(k) A[i, k] * B[k, j]`, plus a separate schedule that says how to reorder, tile or fuse the loops. Compile it to C and measure the speedup.

Goal. Learn loop-nest IR, dependence-preserving loop transformations, and the idea of a scheduling language, which sits at the core of modern ML compilers.

Required features.

- Lexer and parser for tensor declarations with shapes, index-notation statements, element-wise operators, reductions (`sum`, `max`) and common functions (`exp`, `relu`).
- Semantic checks: shape mismatches, unbound or unused indices, reduction over an index that also appears on the left-hand side.
- Lowering to a loop-nest IR, printed in readable form.
- Schedule commands: `reorder`, `tile`, `fuse` (for chains of element-wise operations), `unroll`, and `parallel` (emitting OpenMP pragmas), with checks that reject illegal transformations.

- C code generation, compiled with GCC or Clang, and verified against NumPy for random inputs.
- Benchmarks for matrix multiplication, a 2D convolution and a softmax: naive loops versus your best schedule versus NumPy, at three problem sizes.

Stretch goals. An auto-tuner that searches tile sizes and loop orders automatically; CUDA code generation; a comparison of the same schedule written in TVM or Halide.

Problem-specific deliverables. A language and schedule reference in `docs/language.md`, the generated C code for each benchmark in `examples/generated/`, and a benchmark table or chart in the README.

Career relevance. “Built a Halide/TVM-style tensor DSL compiler with loop scheduling, achieving measured speedups over naive C.”

4 Track C: Optimization and back end

4.1 Problem 9: DAG Construction and Local Optimization of Basic Blocks

Track: C

Problem statement. Given TAC, partition it into basic blocks, build a DAG for each block, and use the DAG to perform local optimizations. Regenerate optimized code from the DAG and measure the improvement.

Goal. Understand basic blocks, value numbering and why DAGs expose redundancy.

Required features.

- Leader detection and basic-block partitioning.
- DAG construction per block, visualized with node labels and attached identifiers.
- Common subexpression elimination, dead-code elimination, constant folding, copy propagation and algebraic simplification (such as $x * 1$, $x + 0$, and strength reduction of $x * 2$).
- Code regeneration from the DAG, with a heuristic node ordering that reduces temporaries.
- A before/after table of instruction count and temporaries per block.

Stretch goals. Handling of array and pointer assignments that kill DAG nodes; comparison of local value numbering with the DAG method.

Problem-specific deliverables. At least 10 blocks taken from textbook exercises, verified against hand-worked answers.

Career relevance. “Implemented DAG-based local optimizations (CSE, constant folding, DCE) over three-address code.”

4.2 Problem 10: Control Flow Graph and Data-Flow Analysis Framework

Track: C

Problem statement. Build a framework that constructs the control flow graph (CFG) of each function and runs classic data-flow analyses using a generic iterative worklist solver. Use the results to report variable lifetimes and dependencies.

Goal. Learn the lattice-based data-flow framework shared by all modern optimizers.

Required features.

- CFG construction from TAC or from a Python or C subset, with Graphviz output.
- A generic solver parameterized by direction, meet operator and transfer function.
- Four analyses built on it: reaching definitions, live variables, available expressions and very busy expressions.
- Dominator tree and natural-loop detection.
- A variable lifetime report: live ranges per variable, dead stores, use-before-definition warnings, and a def-use chain graph.

Stretch goals. Loop-invariant code motion using the analyses; constant propagation over a three-level lattice.

Problem-specific deliverables. For each analysis, the README shows the IN/OUT sets for one example program and the number of iterations needed to reach the fixed point. The README should also briefly discuss why detecting all infinite loops is undecidable.

Career relevance. “Built a generic data-flow analysis framework (liveness, reaching definitions, dominators) with CFG visualization.”

4.3 Problem 11: Register Allocation by Graph Coloring

Track: C

Problem statement. Implement Chaitin–Briggs register allocation. From liveness information, build the interference graph, color it with k registers, and insert spill code when coloring fails. Compare the result with a linear-scan allocator.

Goal. Understand how live ranges become registers, and the trade-off between allocation quality and compile time.

Required features.

- Liveness analysis and interference-graph construction from TAC or a simple RISC-like IR.
- Simplify, spill and select phases, with an animation of the node stack.
- Move coalescing (conservative Briggs or George test).
- Spill-code insertion, re-running allocation until it succeeds.
- A linear-scan allocator for comparison: spills and running time on the same inputs for $k = 2$ to 8.

Stretch goals. Emit runnable RISC-V assembly and run it on a simulator such as RARS or Spike; spill-cost heuristics based on loop depth.

Problem-specific deliverables. A table or chart of spill count versus k for both allocators on at least 8 programs.

Career relevance. “Implemented Chaitin–Briggs graph-coloring register allocation with coalescing, compared against linear scan.”

4.4 Problem 12: SSA Construction and SSA-Based Optimization

Track: C

Problem statement. Convert a CFG into Static Single Assignment (SSA) form using dominance frontiers, run SSA-based optimizations, and convert back out of SSA. This is the IR style used by LLVM, GCC and most research compilers.

Goal. Learn dominance frontiers, ϕ -function placement, variable renaming, and why SSA simplifies optimization.

Required features.

- Dominator tree (Lengauer–Tarjan or the iterative Cooper–Harvey–Kennedy algorithm) and dominance frontiers.
- Minimal or pruned ϕ placement and renaming.
- Sparse conditional constant propagation and dead-code elimination on SSA.
- Out-of-SSA translation with copy insertion.
- Before/after CFG drawings.

Stretch goals. Global value numbering; comparison of your output with `clang -O1 -emit-llvm` on equivalent C code, with a discussion of the differences.

Problem-specific deliverables. A short design note in `docs/` explaining one example end to end.

Career relevance. “Implemented SSA construction and SCCP, mirroring LLVM’s `mem2reg` and `SCCP` passes.”

5 Track D: Static analysis and code metrics

These problems work on real source code (Python via the `ast` module, or C, Java or JavaScript via `tree-sitter`). Using an existing parser is allowed in this track, because the learning target is analysis over the AST, not parsing.

5.1 Problem 13: Code Complexity and Maintainability Analyzer

Track: D

Problem statement. Build a command-line and web tool that computes per-function and per-file quality metrics for a codebase and flags hotspots: cyclomatic complexity, cognitive complexity, Halstead metrics, nesting depth and the Maintainability Index.

Goal. Understand how CFGs and ASTs turn into measurable software-quality metrics, and how the metrics differ.

Required features.

- Cyclomatic complexity from the CFG ($V(G) = E - N + 2P$) and from decision-point counting, showing that both agree.
- Cognitive complexity following the SonarSource specification (nesting increments, no penalty for `switch` shorthand).
- Halstead volume, difficulty, effort and estimated bugs from operator and operand counts.
- Maximum nesting depth, function length, parameter count, and a simple identifier-naming quality check.
- Maintainability Index with thresholds, and a colored HTML hotspot report.
- Runs on at least three open-source repositories, with your numbers compared against `radon` or `lizard`.

Stretch goals. A GitHub Action that comments metric changes on pull requests; a trend chart across a repository’s git history.

Problem-specific deliverables. The README explains each formula with a worked example function and discusses one case where cyclomatic and cognitive complexity disagree.

Career relevance. “Built a code-quality analyzer (cyclomatic, cognitive, Halstead, MI) running as a GitHub Action.”

5.2 Problem 14: AST-Based Code Clone and Plagiarism Detector

Track: D

Problem statement. Build a detector that finds duplicated code within a project and similarity between student submissions, robust to renaming, reformatting and reordering of statements.

Goal. Learn clone types 1 to 4 and compare token-based, AST-based and fingerprint-based detection.

Required features.

- Normalization: strip comments and whitespace, and rename identifiers and literals canonically.
- Token-based detection with winnowing fingerprints (the MOSS algorithm).
- AST-based detection using subtree hashing for Type-2 and Type-3 clones.
- A pairwise similarity matrix shown as a heatmap, and a side-by-side view of matched regions.
- An evaluation set of 10 original programs and 30 disguised copies (renamed, reordered, dead code added), with precision and recall reported per method.

Stretch goals. CFG or program-dependence-graph comparison for Type-4 (semantic) clones; comparison with JPlag.

Problem-specific deliverables. The evaluation set committed with ground-truth labels.

Career relevance. “Designed an AST and fingerprint-based plagiarism detector with measured precision and recall.”

5.3 Problem 15: AST-Based Security Analyzer with Taint Tracking

Track: D

Problem statement. Build a static application security testing (SAST) tool for Python or JavaScript. It detects dangerous patterns and tracks untrusted input (taint) from sources to sinks within functions.

Goal. Apply data-flow analysis to security, the way tools such as Bandit, Semgrep and CodeQL do.

Required features.

- Pattern rules: `eval/exec`, shell commands with `shell=True`, hard-coded secrets, weak hashes (MD5, SHA-1), insecure deserialization (`pickle.loads`).
- Intraprocedural taint analysis with sources (request parameters, `input()`), sanitizers, and sinks (SQL queries, OS commands, file paths).
- A report with CWE identifier, severity, line, and the taint path from source to sink.
- SARIF output so results appear in GitHub code scanning.
- A test suite of at least 20 vulnerable and 20 safe snippets, with false positives and false negatives reported.

Stretch goals. Interprocedural taint through a call graph; comparison with Bandit on the same code.

Problem-specific deliverables. Rules defined in a YAML file, so that adding a rule needs no code change.

Career relevance. “Built a SAST tool with taint analysis and SARIF output integrated into GitHub code scanning.”

5.4 Problem 16: AST Query Engine (a Mini CodeQL / Semgrep)

Track: D

Problem statement. Design a small query language for searching code structurally, for example “find functions with more than three nested loops” or “find calls to `open` that are not inside a `with` block”. Build the engine that parses a query and evaluates it against ASTs.

Goal. Design a domain-specific language end to end (grammar, parser, semantic checks, evaluator) and apply it to program analysis.

Required features.

- A grammar for the query language: node types, attribute predicates, parent/ancestor/descendant relations, logical operators and metavariables such as `$X`.
- A query parser with helpful error messages.
- An evaluator over Python ASTs (or several languages via tree-sitter).
- At least 15 ready-made queries for common bugs and code smells.
- Results reported as file, line and matched snippet, with JSON output.

Stretch goals. A pattern syntax written as code with holes (like Semgrep’s `foo($X, ...)`); automatic rewrite rules (find-and-replace on ASTs).

Problem-specific deliverables. The query language grammar given in EBNF in the README.

Career relevance. “Designed a structural code-search DSL and query engine over ASTs.”

5.5 Problem 17: Call Graph Generator and Dead Code Finder

Track: D

Problem statement. Build a tool that constructs the call graph of a multi-file project, visualizes it, and uses it to find unreachable functions, recursion cycles and the most central functions.

Goal. Learn call-graph construction, the difficulty posed by dynamic dispatch, and graph algorithms applied to programs.

Required features.

- Multi-file call-graph construction for Python, C or Java, resolving imports.
- Handling of methods through class-hierarchy analysis for virtual or overridden calls, with its imprecision documented.
- An interactive graph view with zoom, search and collapse by module.
- Reports: functions unreachable from entry points, recursive cycles (strongly connected components), fan-in and fan-out, and most-called functions.
- A run on one real open-source project of at least 5,000 lines.

Stretch goals. Rapid type analysis to prune impossible targets; comparison of the static graph with a dynamic one recorded by a profiler.

Problem-specific deliverables. A section discussing false and missed edges found in the real-project run.

Career relevance. “Built a multi-file call-graph analyzer with dead-code and recursion detection.”

6 Track E: Quantum compilation with OpenQASM

Quantum compilers apply the same ideas as classical ones (lexing, parsing, IR, peephole optimization, resource allocation) to quantum circuits, and form an active research area. No physics background is needed beyond treating gates as matrices. Qiskit or Cirq may be used *only* to simulate circuits and check your results, never to do the work the problem asks for.

Suggested reading for this track. The OpenQASM 3 specification (<https://openqasm.com>); the Qiskit transpiler documentation; Y. Nam et al., “Automated optimization of large quantum circuits with continuous parameters” (2018); G. Li, Y. Ding and Y. Xie, “Tackling the Qubit Mapping Problem for NISQ-Era Quantum Devices” (the SABRE paper, 2019).

6.1 Problem 18: OpenQASM Front End: Lexer, Parser and Semantic Checker

Track: E

Problem statement. Build a front end for a well-defined subset of OpenQASM 3 (with OpenQASM 2 compatibility). It tokenizes and parses QASM programs, builds an AST, checks semantics, and lowers the program to a flat gate-list IR that Problems 19 to 21 can reuse.

Goal. Apply the full classical front-end pipeline to a real, standardized quantum language.

Required features.

- Lexer and hand-written or generated parser for: the OPENQASM version header, `include`, `qubit/bit` (and `qreg/creg`) declarations, built-in and user-defined `gate` definitions with parameters, gate calls, `measure`, `reset`, `barrier`, and parameter expressions using `pi`.
- Semantic checks: undeclared registers, index out of range, wrong arity, wrong parameter count, the same qubit used twice in one gate, recursive gate definitions.
- Gate-definition inlining (macro expansion) down to a basic gate set.
- Output: the AST as JSON, the circuit drawn as text or SVG, and circuit statistics (qubits, gates by type, depth).
- Verification: for circuits of up to 8 qubits, simulate your IR with a small state-vector simulator (your own or Qiskit’s) and compare with Qiskit’s parse of the same file.

Stretch goals. Classical control (`if`, `for`, `while`) with loop unrolling when bounds are constant; round-trip printing back to valid QASM.

Problem-specific deliverables. The supported grammar subset in EBNF, and at least 20 QASM test files (including small circuits from the QASMBench suite).

Career relevance. “Built an OpenQASM 3 compiler front end with semantic checking, verified against Qiskit.”

6.2 Problem 19: Quantum Circuit Optimizer (Peephole and DAG-Based)

Track: E

Problem statement. Build an optimizer that reduces the gate count and depth of quantum circuits while preserving the unitary. Represent circuits as a DAG of gates on qubit wires (the same idea as DAGs for basic blocks) and apply rewrite rules.

Goal. Transfer classical optimizations (peephole, algebraic simplification, commutation-based reordering) to a quantum IR, and measure their effect honestly.

Required features.

- Circuit DAG construction from QASM (using Problem 18 or a simple parser).
- Passes: cancellation of adjacent inverse pairs ($H-H$, $CX-CX$, $S-S^\dagger$), rotation merging ($R_z(a)R_z(b) = R_z(a+b)$), removal of identity rotations, commutation rules that bring cancellable gates together, and single-qubit gate fusion.
- A pass manager that runs passes to a fixed point and logs the effect of each pass.
- Correctness check: unitaries of input and output compared (up to global phase) for circuits of up to 10 qubits.
- A benchmark on at least 15 circuits (QASMBench or RevLib): gate count, CX count, T count and depth before and after, compared with Qiskit's transpiler at optimization levels 1 and 3.

Stretch goals. ZX-calculus simplification with PyZX as a comparison baseline; template matching from a small library of circuit identities.

Problem-specific deliverables. A benchmark table and chart in the README, with a short discussion of which passes mattered most.

Career relevance. “Developed a DAG-based quantum circuit optimizer, benchmarked against Qiskit's transpiler.”

6.3 Problem 20: Qubit Mapping and Routing on Hardware Coupling Graphs

Track: E

Problem statement. Real quantum chips only allow two-qubit gates between physically connected qubits. Build a compiler pass that maps logical qubits to physical ones and inserts SWAP gates so that every two-qubit gate is legal, while minimizing the added SWAPs and depth. This is the quantum counterpart of register allocation.

Goal. Understand constrained resource allocation as a graph problem, and implement a published heuristic.

Required features.

- Coupling graphs as JSON input: line, ring, 2D grid, heavy-hex, and one real device topology.
- Initial placement: a trivial mapping and a greedy mapping based on the interaction graph.
- Routing: a naive shortest-path SWAP router and the SABRE look-ahead heuristic with bidirectional passes.
- Output: routed QASM, with verification that every two-qubit gate respects the coupling graph and that the circuit is equivalent.
- Results: SWAP count and depth overhead for each router on at least 10 benchmark circuits and 3 topologies, compared with Qiskit's SabreSwap.

Stretch goals. An exact solver (SAT or integer programming) for small circuits to measure the gap between the heuristic and the optimum; noise-aware mapping using per-edge error rates.

Problem-specific deliverables. An animation or step view of the mapping as SWAPs are inserted.

Career relevance. “Implemented the SABRE qubit-routing algorithm and evaluated it on heavy-hex and grid topologies.”

6.4 Problem 21: A Small High-Level Quantum Language Compiled to OpenQASM

Track: E

Problem statement. Design a tiny high-level quantum language with named registers, loops, functions, controlled operations and simple reversible arithmetic. Write a compiler that translates it into OpenQASM for a chosen native gate set and reports resource estimates.

Goal. Build a complete compiler pipeline (lexer, parser, semantic checks, IR, lowering, code generation) whose target is quantum.

Required features.

- Language grammar and parser, with the semantic rules that qubits cannot be copied (no-cloning) and are not reused after measurement without `reset`.
- Constructs: `for` with constant bounds (unrolled), functions (inlined), a `ctrl` modifier, the inverse (`adj`) of a function obtained by reversing and inverting its gates, and an in-place adder on qubit registers.
- Decomposition passes: multi-controlled X into Toffolis with ancillas, Toffoli into Clifford+T, and arbitrary single-qubit gates into a target basis (for example $\{R_z, \sqrt{X}, X, CX\}$).
- A resource estimator reporting qubits, ancillas, CX count, T count and depth.
- Demonstration programs: Bell and GHZ states, Deutsch–Jozsa, Grover search on 3 qubits, and the quantum Fourier transform, with simulated outputs matching the expected distributions.

Stretch goals. Automatic uncomputation of ancillas; passing the output through the optimizer of Problem 19.

Problem-specific deliverables. A language reference in `docs/language.md` with the grammar and one fully compiled example.

Career relevance. “Designed a quantum programming language and compiler targeting OpenQASM with Clifford+T resource estimation.”

7 Track F: Visual and educational compiler tooling

7.1 Problem 22: Compiler Pipeline Explorer with Animated Lessons and Adaptive Quizzes

Track: F

Problem statement. Build a web-based learning platform where a student types a short program and watches it move through every compiler phase: tokens, parse tree, AST, symbol table, three-address code, optimized code and target code. Pair it with animated explanations of LL(1) and LR(1) parsing made with Manim, and quizzes generated from the student’s own inputs.

Goal. Combine several course topics into one product, and practise building educational software that other students will actually use.

Required features.

- A pipeline view with each phase in its own panel, and hover-linking between a source token and its AST node, TAC lines and target instructions.
- At least two Manim videos (3–5 minutes each): an LL(1) parse and an LR(1) parse of the same string on the same grammar, contrasting top-down and bottom-up parsing.
- A quiz engine that generates questions automatically (“What is $\text{FIRST}(T)$?”, “Fill this LL(1) table cell”, “Shift or reduce in this state?”, “How many basic blocks?”) with instant feedback and explanations.
- Progress tracking per topic, with harder questions after correct answers.
- A usability test with at least 5 classmates: a short survey and a before/after quiz score comparison.

Stretch goals. A “compiler as a story” mode in which each phase is a character narrating what it does; public deployment on GitHub Pages or Vercel.

Problem-specific deliverables. A live demo link, Manim source scripts in `animations/`, and the usability results in the README. Intended for a pair: one student on the pipeline and quizzes, one on the animations.

Career relevance. “Built and deployed an interactive compiler-learning platform with Manim animations and auto-generated quizzes, tested with student users.”

8 Submission guidelines

All submissions are made through the course repository:

`https://github.com/venkyjntu-git/compiler-design-assignments-2026-27`

Put your complete project in the folder for your problem, `submissions/P<nn>/`, where `<nn>` is the two-digit problem number (for example, Problem 7 goes in `submissions/P07/`). Copy `templates/README_TEMPLATE.md` into that folder as `README.md`. Submit before **28 October 2026, 23:59 IST**.

8.1 General rules

- **Language.** Any mainstream language (Python, C++, Java, Rust, TypeScript, and so on). Parser generators (ANTLR, Bison, PLY and similar) are allowed only where the problem says so; the core algorithm the problem teaches must be your own code.
- **Tests.** Tests must run with one command (`pytest`, `npm test`, `cargo test`, ...) documented in the README. Aim for at least 70% coverage of the core logic.
- **Reproducibility.** Installation steps must work on a clean machine. A Dockerfile is welcome.
- **Demo.** A 3–5 minute screen recording linked in the README, and at least three screenshots or GIFs in `docs/`.
- **Academic integrity.** Cite every library, paper, tutorial and AI assistant you used in the *Credits* section. Uncredited copied code is treated as plagiarism. You must be able to explain every line of your code at the viva.

8.2 README template

Your `README.md` must contain the following sections, in this order.

```
# <Project name>
One-line description of what the tool does.
! [demo] (docs/demo.gif)

## Team
Name, roll number (and partner), problem number, course, semester.

## Problem statement
What problem this solves and why it matters (3–5 sentences).

## Background theory
The compiler concept behind the tool, with one fully worked example
(e.g. FIRST/FOLLOW sets computed by hand for a small grammar).

## Features
```

```
- Required features implemented (tick each one)
- Stretch goals implemented

## Architecture
Diagram of the main modules and data flow; key data structures.

## Installation
Step-by-step commands for a clean machine.

## Usage
CLI commands or UI walkthrough with example input and output.

## Examples and results
Sample runs, benchmark tables, comparison with a reference tool.

## Testing
How to run the tests and what they cover.

## Limitations and future work
What does not work yet and how you would extend it.

## Demo video
Link.

## Credits and references
Textbook chapters, papers, libraries, AI assistance used.
```

8.3 Evaluation rubric

Component	Marks	What earns full marks
Correctness of core algorithm	2	Matches textbook results on all given test cases; edge cases handled.
Features and usability	2	All required features work; clear CLI or UI; helpful error messages.
README and documentation	2	Follows the template; theory explained with a worked example.
Evaluation and results	2	Benchmarks, sample outputs or comparison with a reference tool.
Demo and viva	2	Student explains the algorithm and design choices without notes.